

KMC Project Architecture & Workflow

This document provides a comprehensive overview of the KMC (Kisan mithar consultancy) project, detailing its architecture, tech stack, folder structure, and operational workflows.

1. Project Overview

KMC is a full-stack web application designed to empower farmers by providing essential agricultural services, information, and a marketplace for farming supplies. It serves as a centralized hub for:

- **Expert Advice:** Soil testing, crop selection, and advisor consultations.
- **Resource Management:** Marketplace for fertilizers and farming equipment.
- **Information Hub:** Agricultural blogs, success stories, and real-time market prices.
- **Admin Control:** Comprehensive dashboard for managing users, products, and platform data.

The project follows a modern MERN-like architecture with a decoupled frontend and backend, ensuring scalability and ease of maintenance.

1.1 Problem Statement

The agricultural sector currently faces a "Knowledge-Action Gap." Farmers are often disconnected from the scientific and market realities that dictate profitability. This gap manifests in three critical areas:

- **Information Asymmetry:** Farmers frequently rely on generational knowledge or hearsay rather than scientific data for crop selection and disease management. This leads to preventable crop failures and lower yields.
- **Supply Chain Inefficiencies:** The market for essential inputs like fertilizers and equipment is fragmented. Intermediaries often struggle to provide transparent pricing or quality assurance, driving up costs for the end farmer.
- **Advisory Accessibility:** Access to qualified agricultural scientists (Field Officers) is geographically limited. Farmers cannot easily "book" an expert visit, leaving them unsupported during critical infestation or planting periods.

KMC solves these problems by digitizing the agricultural lifecycle. It is a centralized platform that integrates:

- **Direct-to-Farmer Marketplace:** Eliminating middlemen for fertilizers and equipment.
- **On-Demand Advisory:** A booking system for physical farm visits by experts.
- **Data-Driven Decision Making:** Real-time market prices and weather-aligned crop advice.

1.3 Business Impact

- **Yield Maximization:** Timely expert advice and correct input usage can increase crop yields significantly.
- **Cost Reduction:** Direct sourcing of equipment and fertilizers reduces input costs for farmers.
- **Operational Efficiency:** Digitizing records allows for better policy-making and resource allocation by administrators.

2. Learning Outcomes

By exploring or developing this project, one gains proficiency in:

- **Full-Stack Development:** Integrating a React frontend with a Node/Express backend.
- **Database Designing:** Using Mongoose for complex data modeling and relationships.
- **State Management:** Handling global state and navigation in a multi-page React app.
- **RESTful API Design:** Creating secure and efficient endpoints for CRUD operations.
- **Third-Party Integrations:** Implementing Cloudinary for image hosting, Nodemailer for emails, and payment/SMS gateways.
- **Admin Dashboards:** Building interactive analytics and management interfaces.
- **Security Best Practices:** Implementing JWT-based authentication and role-based access control (RBAC).

3. Tech Stack

Frontend

- **Framework:** [React.js](#) (Vite)
- **Styling:** [Tailwind CSS](#)
- **Icons:** [Lucide React](#), [React Icons](#)
- **State/Routering:** [React Router DOM](#)
- **Charts:** [Recharts](#)
- **Notifications:** [React Toastify](#)
- **HTTP Client:** [Axios](#)

Frontend folder structure

```
KMC/Frontend/
├── public/           # Static assets (favicons, etc.)
├── src/
│   ├── assets/      # Images and icons used in the UI
│   ├── components/  # Reusable UI components (Navbar, Footer, Admin layouts)
│   ├── context/     # React Context for global state management
│   ├── pages/       # Individual page components (Auth, Admin, Farming, etc.)
│   ├── App.jsx      # Main routing configuration
│   └── main.jsx      # Application entry point
├── vite.config.js   # Vite build configuration
└── vercel.json      # Deployment configuration for Vercel
```

Key Directories

- **src/components**: Reusable UI components (Header, Navbar, Footer, etc.)
- **src/pages**: Page components corresponding to routes (Home, Login, Services, etc.)
- **src/context**: React Context for state management (**AppContext.jsx**)
- **src/assets**: Static assets (images, icons)

Routing (App.jsx)

The application defines the following main routes:

- **/**: Home
- **/login**: Login
- **/email-verify**: Email Verification
- **/reset-password**: Reset Password
- **/soil-crop-analysis**: Soil Testing
- **/equipments**: Equipments

- `/packages`: Packages
- `/blogs`: Blogs
- `/about`: About Us
- `/fertilizers`: Fertilizers
- `/orchard-planning`: Orchard Planning
- `/market-prices`: Market Prices
- `/whether-insights`: Weather Insights
- `/crop-selection`: Crop Selection
- `/contact`: Contact Us
- `*`: 404 Not Found

Key Components

`main.jsx` :

Entry point. Wraps the application in `BrowserRouter` and `AppContextProvider`.

`index.css`

Global styles. Imports 'Outfit' font from Google Fonts and initializes TailwindCSS.

`Navbar.jsx` :

- Responsive navigation bar.
- Handles authentication state (`userData`, logout).
- Includes verification OTP sending logic.
- Responsive mobile menu.

`Header.jsx` :

- Hero section with background image carousel.
- Displays "Our Services" grid cards.
- "Drone Technology" promotional section.

Backend

- **Environment:** [Node.js](#)
- **Framework:** [Express.js](#)
- **Database:** [MongoDB](#) (Mongoose ODM)
- **Authentication:** [JSON Web Tokens \(JWT\)](#), [BcryptJS](#)
- **Media Storage:** [Cloudinary](#)
- **Email/SMS:** [Nodemailer](#), [Fast2SMS](#)
- **Middleware:** Multer (file uploads), Cookie Parser, CORS.

Backend folder structure

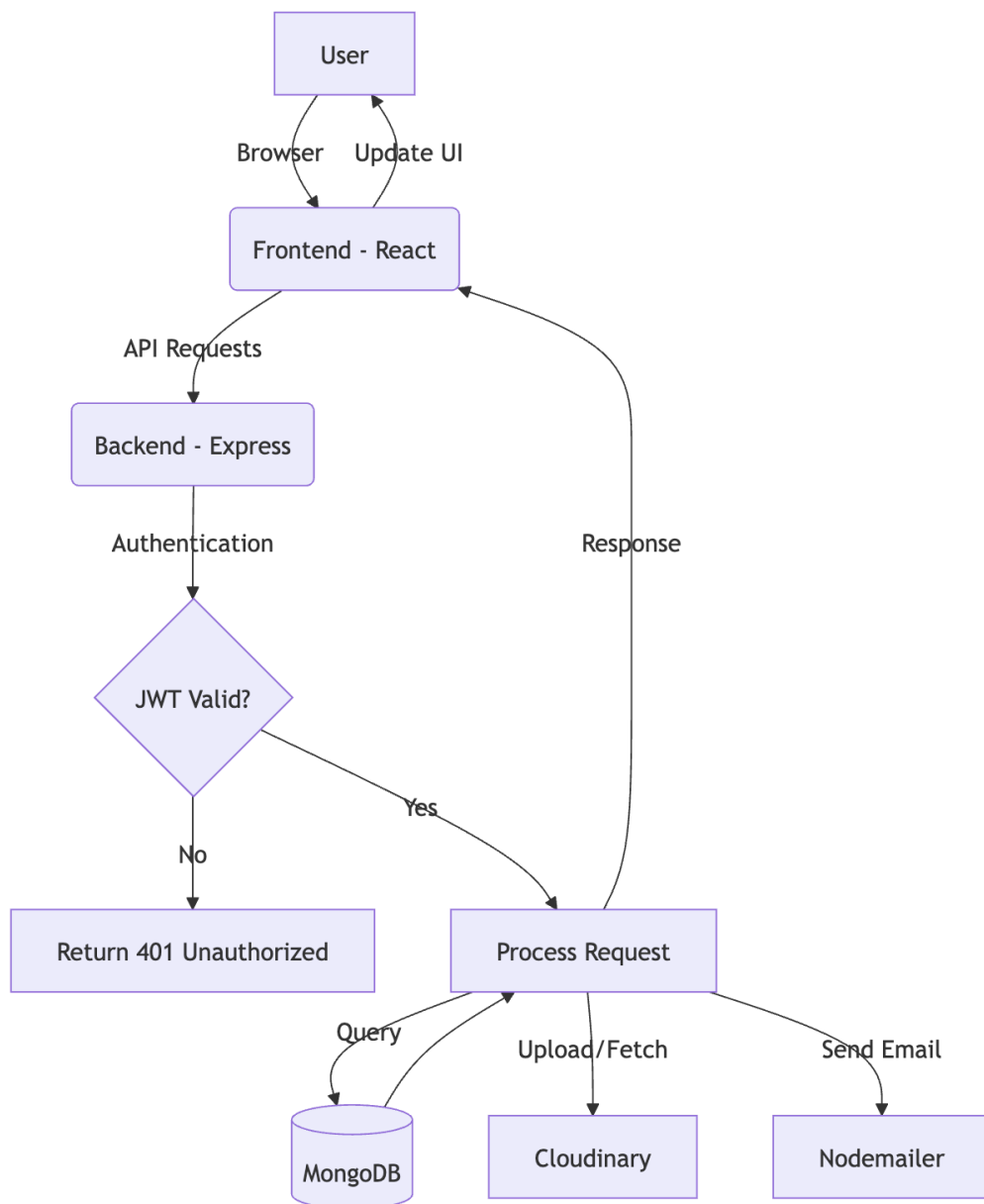
```
KMC/Backend/
├── config/           # Database, Cloudinary, and SMS configurations
├── controllers/      # Logical implementation of API endpoints
├── middleware/       # Authentication and error-handling middleware
├── models/           # Mongoose schemas for MongoDB
├── routes/           # Express route definitions
├── seedData.js       # Scripts to populate the database with initial data
├── server.js         # Main entry point of the server
└── vercel.json       # Deployment configuration for Vercel
```

5. System Architecture & Workflow

5.1 User Interaction Workflow

1. **User Action:** Triggering the request from the React UI.
2. **API Routing:** How Express handles the endpoint.
3. **Authentication:** The role of the `userAuth` middleware.
4. **Business Logic:** Controller validation.
5. **Database:** Mongoose interaction with MongoDB Atlas.
6. **Response:** Updating the UI state.

Workflow



5.2 Admin Panel Architecture

The Admin Panel is a secured, isolated interface designed for system oversight. It uses a strict **Role-Based Access Control (RBAC)** mechanism to ensure only authorized personnel can access sensitive data.

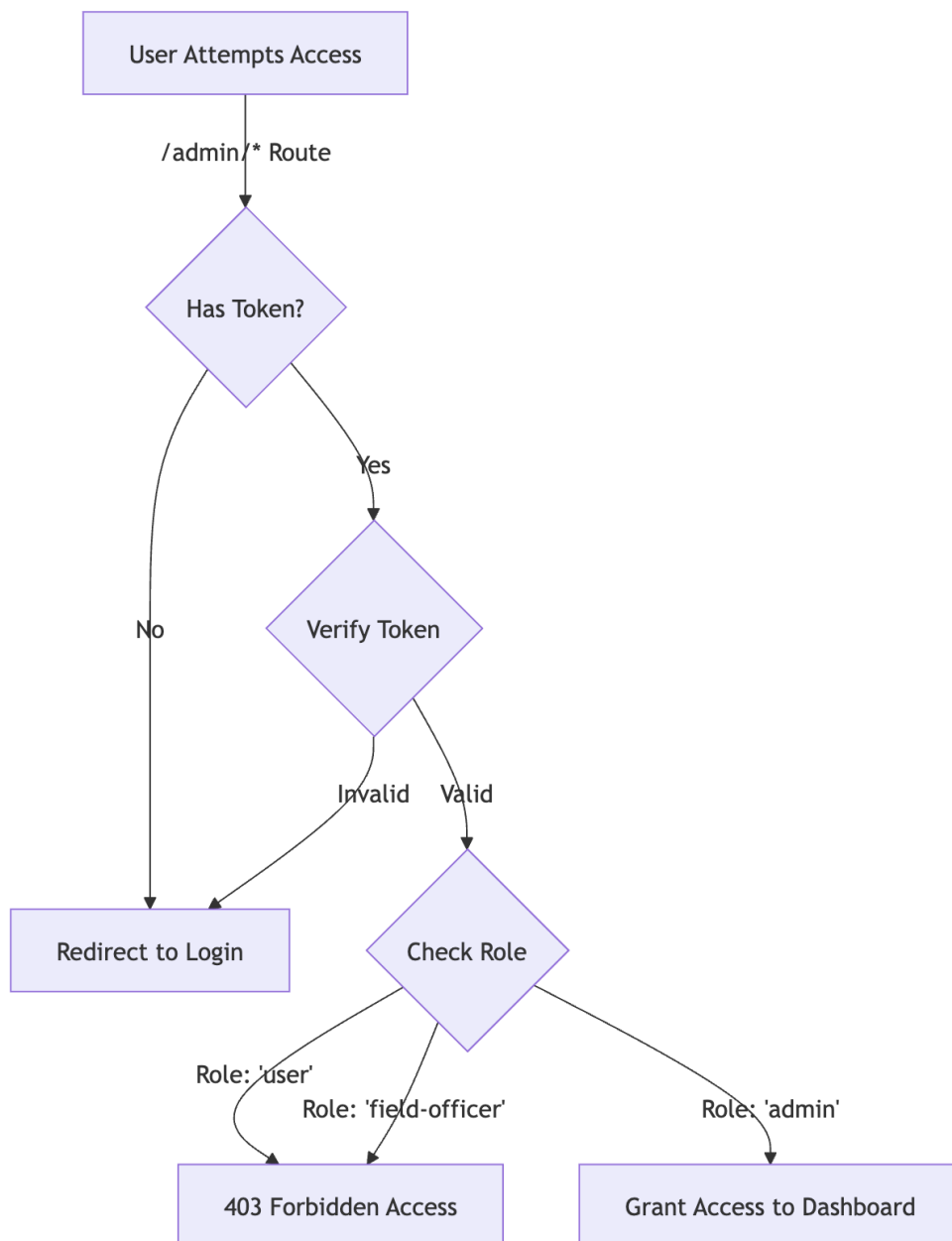
Access Control Logic

Access is governed by the adminAuth middleware, which performs a dual-check:

Authentication: Verifies the JWT token from the `httpOnly` cookie is valid and not expired.

Authorization: Decodes the token to verify `user.role === 'admin'`.

Access Flow Chart



Dashboard Capabilities

Once authenticated, the Admin has exclusive access to:

- **Analytics:** Real-time graphs for revenue, user growth, and crop distribution.
- **User Management:** Approve/Reject new farmer registrations and manage field officers.
- **Content Management:** Publish blogs, success stories, and update market prices.
- **Inventory Control:** Add/Edit/Delete fertilizers and equipment.

5.3. User Roles & Access Control

The system implements strict **Role-Based Access Control (RBAC)** to ensure data security.

Role	Key Capabilities	Key Capabilities
Farmer (User)	Basic	• Browse Products & Blogs • Book Farm Visits • Place Orders • View Market Prices
Field Officer	Intermediate	• View Assigned Bookings • Update Booking Status (Pending → Completed) • Submit Visit Reports
Admin	Full Control	• User Management (Approve/Reject Farmers) • Product Inventory Management • Analytics Dashboard • Manage Content (Blogs, Stories)

6. Operational Workflows

6.1 Farm Visit Booking Cycle

1. **Request:** Farmer logs in, selects a date, and submits a "Request for Visit".
2. **Assignment:** Admin/System assigns a **Field Officer** to the request based on District/Location.
3. **Execution:** Field Officer visits the farm on the scheduled date.
4. **Completion:** Officer marks the booking as "Completed" in the system.

6.2 E-Commerce Order Cycle

1. **Selection:** Farmer adds Fertilizers/Equipment to the cart.
2. **Checkout:** System validates stock availability in **Product** collection.
3. **Creation:** An **Order** document is created with status "Processing".
4. **Fulfillment:** Admin updates status to "Shipped" -> "Delivered".

7. Database Schema Design (Key Models)

User Collection :

javascript

```
{
  name: String,
  email: { type: String, unique: true },
  role: { type: String, enum: ['user', 'admin', 'field-officer'] },
  district: String,
  crops: [String], // Array of crops they grow
  isAccountVerified: Boolean
}
```

Booking Collection :

javascript

```
{
  farmerId: ObjectId (Ref: User),
  visitDate: Date,
  assignedOfficer: ObjectId (Ref: User),
  status: { type: String, enum: ['Pending', 'Confirmed', 'Completed'] },
  notes: String
}
```

The system uses several Mongoose models to manage data:

- **User:** Stores credentials, roles (Admin/User), and profile info.
- **Blog:** Content for agricultural updates with slugs for SEO.
- **Equipment & Fertilizer:** Product details for the marketplace.
- **Order/EquipmentOrder:** Tracking purchases and customer details.

- **MarketPrice:** Real-time or updated pricing for crops.
- **SuccessStory:** Testimonials and case studies.
- **Booking:** Farm visit or consultation schedules.

8. API Documentation Summary

Authentication (`/api/auth`)

- `POST /login`: Validates credentials, sets HTTP-Only cookie.
- `POST /verify-otp`: Multifactor authentication support.

Admin Operations (`/api/admin`)

- `GET /dashboard-stats`: Aggregated data for analytics charts.
- `PATCH /users/:id/role`: Promote users/officers.

Marketplace (`/api/products`)

- `GET /`: List all products (supports pagination `?page=1`).
- `GET /:id`: Product details.

9. How to Run + Quick Tests

Prerequisites

- Node.js (v16 or higher)
- MongoDB account (Atlas or Local)
- Cloudinary Account (for media)

Setup Instructions

1. Clone the Repo: `bash`

```
git clone <repo-url>
cd KMC
```

2. Backend Setup: `bash`

```
cd Backend
npm install
# Create a .env file based on existing .env structure
npm start
```

3. Frontend Setup: `bash`

```
cd ../Frontend
npm install
# Create a .env file with VITE_BACKEND_URL
npm run dev
```

Quick Tests

- **Auth Test:** Create a new account and verify if you can log in.
- **API Health:** Visit `http://localhost:4000/` to check if the server is running.
- **Admin Access:** Log in with an admin account and check if the `/admin/dashboard` loads metrics.

10. Debugging Techniques

- **Console Logging:** Use `console.log` on the backend to trace request payloads (`req.body`) and database results.
- **Network Tab:** Use Chrome DevTools Network tab to inspect failing API calls (check status codes like 404, 500).
- **Redux/Context DevTools:** Inspect state changes if using global state.
- **Mongoose Debug Mode:** Enable `mongoose.set('debug', true)` to see raw MongoDB queries.
- **Tailwind Inspection:** Use the browser inspector to debug styling issues and responsive breakpoints.

11. Security & Deployment

- **JWT Secrets:** All sensitive tokens are stored in environment variables, never committed to VCS.
- **CORS Configuration:** Restricts API access to authorized frontend domains.
- **Deployment:** Both Frontend and Backend are configured for deployment on **Vercel** using `vercel.json` files.
- **Role-Based Access:** Specialized middleware ensures only Admins can access `/api/admin` routes.
- **XSS Protection:** Tokens stored in `HttpOnly` cookies, inaccessible to JavaScript.

- **Input Validation:** All API inputs are validated at the controller level before DB queries.
- **Global Error Handler:** Centralized middleware catches async errors and sends standardized JSON responses.

12. Deployment & Scalability

Deployment

- **Frontend:** Deployed on **Vercel** (Static Site Generation).
- **Backend:** Deployed on **Vercel** (Serverless Functions) or Node.js hosting.
- **Database:** Hosted on **MongoDB Atlas** (Managed Cloud Database).

Scalability

- **Stateless API:** The backend is RESTful and stateless, allowing easy horizontal scaling.
- **CDN:** Images and static assets are delivered via global CDNs (Cloudinary/Vercel Edge).